158.337 Database Development

Week 8 – Procedural Language SQL (contd.)

Cursors and triggers are exciting things, and are advanced concepts in PL/SQL.

What are **cursors**? When stored procedures or PL/SQL blocks return more than a single value, we use cursors. We know that mostly SQL statements return many rows (depending upon the query). For instance – if, we have to write a SQL query which lists all those students enrolled who are more than 16 years of age. This would return more than one row. In such a case, we have to create explicit cursors.

A **cursor** is used to hold the output of a SQL statement that may return two or more rows (but could return zero rows or only one). The cursor processing commands are OPEN, FETCH and CLOSE anywhere between the BEGIN and END keywords of the PL/SQL block. However, using a FOR loop makes this task much easier. Actions including creating a record variable, opening the cursor, looping through one row at a time until the last row is retrieved from the cursor and then closing the cursor are done automatically. Life gets simple ;). But you need to know both types of cursor movements – since FETCH statement performs more efficiently than the FOR statement especially when performing bulk operations, hence we have asked you to implement both types in the tutorial exercise.

And what is a **trigger**? A trigger is a block of PL/SQL code that is automatically invoked by the DBMS upon the occurrence of a data manipulation event (INSERT, UPDATE or DELETE.) Triggers are always associated with a table and are invoked before or after a data row is inserted, updated, or deleted. Any table can have one or more triggers.

Triggers provide a method of enforcing business rules such as:

- A customer making a credit purchase must have an active account.
- A student taking a class with a prerequisite must have completed that prerequisite with a B grade.

Note that one statement can end up firing off a number of triggers. Not only can we have multiple triggers attached to a single table, but a trigger can perform DML operations that may affect other tables and fire additional triggers. These could lead to table mutating errors…Therefore, it is important to make our code clear and maintainable and map the flow of triggers properly.

Lastly, I have also given you an example of Exception Handling in Triggers. It is important to safeguard your program from unforeseen errors.

Next week we will cover another language for writing queries (LINQ). Times sure are changing!